

EDUCAR PARA TRANSFORMAR

Plan operativo revisado de trabajo en paralelo

Web + Supabase | Dos desarrolladores | Orden lógico por dependencias

Decisión principal de esta versión

Se reordena el trabajo para desarrollar primero las funcionalidades que crean los datos base y después las que dependen de ellos. El nuevo orden operativo reemplaza el orden anterior de los Sprints y deberá reflejarse también en el Plan de Trabajo académico.

Equipo: Martín Moschen - Claudio Gómez

Metodología de Sistemas II - TUP 2026

Versión revisada: 26/08/2026

1. Qué cambia y por qué

Problema detectado en el orden anterior

El Plan ubicaba primero la inscripción a deporte y transporte. Pero ambas necesitan alumnos existentes. Además, la inscripción deportiva necesita deporte, grupo, horario y profesor responsable. Por eso el orden anterior obligaba a usar mocks o seeds antes de contar con las entidades reales.

HU / RF	Qué hace	Dependencia real
HU-01 / RF-01	Crea una solicitud de inscripción en estado pendiente.	Por sí sola NO crea un alumno activo. Se necesita RF-04 para aprobar/habilitar.
HU-02 / RF-10	Inscribe un alumno a deporte.	Necesita alumno activo + RF-08 deporte + RF-09 grupo + RF-19 horario + profesor responsable.
HU-05 / RF-11	Inscribe un alumno a transporte.	Necesita alumno activo + los 4 recorridos.
HU-03 / RF-20	Padre inscribe hijo al cursado.	Necesita RF-14 padre-hijo + RF-15 acceso/rol + RF-17 curso + RF-18 materia.
HU-06 / RF-22	Consulta reportes.	Depende de que ya existan datos de alumnos, docentes, cursos, deportes y servicios.

Nuevo criterio

Primero se implementan las funcionalidades que generan o habilitan datos base. Luego las inscripciones y, por último, los reportes. Si una dependencia aún no está disponible, se usa un MOCK o SEED explícitamente documentado y con una tarea de eliminación.

2. Nuevo orden operativo

Etapas	Objetivo	Trabajo principal
Etapas 0	Fundación técnica	Supabase, Auth base, contratos y catálogos mínimos.
Sprint 1	Crear datos base	Martín: HU-01/RF-01 + RF-04 mínimo. Claudio: HU-04/RF-05 + RF-13/RF-15 base.
Sprint 2	Inscripciones de servicios	Martín: RF-08/RF-09/RF-19 + HU-02/RF-10. Claudio: HU-05/RF-11.
Sprint 3	Cursado y reportes	Martín: RF-14/RF-18 + HU-03/RF-20. Claudio: RF-21 + HU-06/RF-22.
Cierre	Hardening y release	Eliminar mocks, RLS, pruebas, correcciones y deploy.

Resultado: las seis HU prioritarias siguen repartidas 3 y 3, pero ahora el orden respeta las dependencias del dominio.

3. Reparto completo de requerimientos

Cada RF tiene un dueño principal. El dueño implementa frontend, persistencia de su dominio, validaciones y pruebas. El compañero revisa mediante Pull Request.

RF	Dominio	Responsable
RF-01 a RF-04	Alumnos: solicitud, consulta, modificación, aprobación/estado	Martín
RF-05 a RF-07	Profesores: registro, consulta y modificación	Claudio
RF-08 a RF-10	Deportes, grupos e inscripción deportiva	Martín
RF-11	Transporte	Claudio
RF-12	Comedor	Claudio
RF-13	Usuarios	Claudio
RF-14	Asociación padre-hijo	Martín
RF-15	Login, roles y permisos	Claudio
RF-16 a RF-20	Nivel, curso, materia, horarios e inscripción al cursado	Martín
RF-21	Asignación profesor-materia-curso	Claudio
RF-22	Reportes	Claudio
RF-23	Chatbot de recomendación deportiva	Martín

3.1 Responsabilidad transversal

Quién	Zona principal	Regla
Martín	Alumno + Académico + Deportes	Mantiene contratos y reglas de sus dominios; crea migraciones nuevas de esos dominios.
Claudio	Auth + Profesores + Servicios + Reportes	Inicializa Supabase, mantiene autenticación/roles base y sus migraciones de dominio.
Ambos	Archivos compartidos	App.tsx, package.json, src/shared/**, contratos comunes y config global: cambio revisado por el otro.

Regla de propiedad

Un RF puede necesitar datos de otro RF, pero eso no cambia su dueño. Si la dependencia no está lista, se usa un contrato + mock/seed temporal; no se toma código del compañero sin handoff.

4. Etapa 0 - Fundación técnica

Responsable	Issue de GitHub	Salida esperada
Claudio	[SETUP] Inicializar Supabase en educar-servidor	supabase/ versionado, .gitignore, README y proyecto enlazado.
Claudio	[AUTH][RF-13/RF-15] Perfiles, roles y sesión base	profiles/roles + Supabase Auth + conexión desde educar-cliente.
Martín	[CORE][RF-16/RF-17/RF-19] Catálogos académicos mínimos	niveles, cursos y horarios mínimos en BD + datos SEED controlados.
Martín	[CONTRACT] Tipos compartidos de Alumno/Curso/Materia/Horario	interfaces estables para cliente y servidor.
Ambos	[SHARED] Congelar contratos comunes	30-45 min: nombres, IDs, estados y relaciones que ambos consumen.

Preferencia de datos temporales

Una vez que Supabase esté disponible, preferir SEED en la base real antes que MOCK en el frontend. El SEED prueba el esquema real; el MOCK solo se usa para desbloquear una pantalla cuando todavía falta la dependencia.

5. Sprint 1 - Crear datos base

Objetivo

Al terminar este Sprint deben existir alumnos habilitados para operar y profesores reales. Esto elimina las dos dependencias principales de deporte, transporte y reportes.

5.1 Martín - HU-01 / RF-01 + RF-04 mínimo

Tarjeta	Debe quedar
[HU-01][RF-01] Solicitud pública de inscripción	Formulario, validaciones, duplicado por DNI y estado PENDIENTE en Supabase.
[RF-04] Aprobación/activación mínima de alumno	Administración puede aprobar la solicitud y generar/habilitar el alumno activo.
[TEST] Flujo solicitud -> aprobación	Caso válido, faltantes, duplicado y aprobación.

5.2 Claudio - HU-04 / RF-05 + Auth base

Tarjeta	Debe quedar
[HU-04][RF-05] Registrar profesor	Profesor persistido con legajo, DNI, datos, especialidad y estado.
[RF-13/RF-15] Auth/roles mínimo	Usuario autenticado, rol identificado y rutas privadas protegidas.

Tarjeta	Debe quedar
[TEST] Profesor + permisos	Registro válido, datos faltantes, cancelación y acceso por rol.

Inconsistencia resuelta

HU-01 crea una solicitud, no un alumno activo. Por eso RF-04 mínimo entra en este Sprint aunque no sea una de las seis HU priorizadas. Sin ese paso, HU-02 y HU-05 seguirían dependiendo de alumnos seed.

6. Sprint 2 - Deportes y transporte

6.1 Martín - HU-02 / RF-10

Tarjeta	Debe quedar
[RF-08] Deportes mínimos	Deportes disponibles con profesor responsable.
[RF-09] Grupos deportivos	Grupo por nivel y horario.
[RF-19] Horarios necesarios para deporte	Datos suficientes para detectar superposición.
[HU-02][RF-10] Inscripción a deporte	Máximo 2 deportes, sin choque horario y sin duplicados.

6.2 Claudio - HU-05 / RF-11

Tarjeta	Debe quedar
[HU-05][RF-11] Transporte	Cuatro recorridos cargados y alumno asociado a un recorrido.
[RULES] Validaciones transporte	Alumno válido, recorrido obligatorio y sin duplicado.
[TEST] Transporte	Probar los 4 recorridos, caso válido y sin recorrido.

Mock permitido solo como contingencia

Si Sprint 1 no dejó alumnos/profesores disponibles a tiempo, se puede usar SEED temporal en Supabase. El Issue debe marcarse como datos temporales y crear una tarea explícita para reemplazarlos. No se considera Done final mientras dependa de un mock requerido por producción.

7. Sprint 3 - Cursado y reportes

7.1 Martín - HU-03 / RF-20

Tarjeta	Debe quedar
[RF-14] Asociación padre-hijo	Un padre puede tener varios hijos y solo operar sobre los propios.
[RF-18] Materias mínimas	Materias asociables al curso del alumno.

Tarjeta	Debe quedar
[HU-03][RF-20] Inscripción al cursado	Padre selecciona hijo y materia de su curso; sin duplicados.
[TEST] Cursado y autorización	Cursado válido, hijo no asociado y materia duplicada.

7.2 Claudio - HU-06 / RF-22

Tarjeta	Debe quedar
[RF-21] Profesor-materia-curso mínimo	Relación necesaria para consultas académicas y reportes.
[HU-06][RF-22] Reportes priorizados	Tipo de reporte, criterio, resultados, criterio faltante y sin datos.
[TEST] Reportes	Consultas con datos reales/seed controlado y permisos de administrador.

Por qué reportes va último

RF-22 depende de casi todos los dominios. Antes de este Sprint ya deben existir alumnos, profesores, cursos, materias, deportes y transporte. Se pueden usar SEED para ampliar el volumen de prueba, pero las consultas y relaciones deben ser reales.

8. Política simple de MOCK y SEED

Tipo	Qué es	Regla
MOCK	Dato falso en frontend/memoria	Solo para desbloquear UI cuando aún no existe la dependencia. Debe etiquetarse mock-temporal.
SEED	Dato de prueba insertado en Supabase	Preferido para probar integración real. Puede permanecer como dato de demo controlado.
REAL	Dato creado por el flujo productivo	Objetivo final para todas las HU de entrega.

Formato obligatorio en el Issue

Datos temporales: MOCK / SEED / NO. Motivo. Qué dependencia falta. Qué tarjeta lo elimina. Una HU productiva no pasa a Done final si todavía depende de un MOCK.

9. GitHub Projects - tablero Kanban recomendado

Usar un único Project para cliente y servidor. La app móvil queda en Backlog hasta octubre.

Status	Uso
Backlog	Todavía no se trabaja.

Status	Uso
Ready	Tiene dueño, criterios, dependencias resueltas o mock/seed documentado.
In Progress	Una sola tarjeta activa por persona.
Review	PR abierto; revisa el compañero.
Testing	Pruebas funcionales/integración.
Done	Mergeado, probado y sin mock productivo pendiente.

9.1 Campos mínimos

Campo	Valores
Responsable	Martín Claudio Compartido
Sprint	Fundación Sprint 1 Sprint 2 Sprint 3 Cierre Octubre
RF/HU	Ej.: HU-01 / RF-01
Repo	educar-cliente educar-servidor
Tipo	Feature BD/RLS Test Bug Shared
Datos temporales	NO MOCK SEED

10. Tarjetas exactas para cargar en GitHub Projects

FUNDACIÓN

Responsable	Issue	RF/HU
Claudio	[SETUP] Inicializar Supabase en educar-servidor	N/A
Claudio	[RF-13/RF-15] Auth, perfiles y roles base	RF-13 / RF-15
Martín	[RF-16/RF-17/RF-19] Catálogos académicos mínimos + SEED	RF-16 / RF-17 / RF-19
Martín	[CONTRACT] Alumno / Curso / Materia / Horario	N/A
Compartido	[SHARED] Congelar contratos comunes	N/A

SPRINT 1

Responsable	Issue	RF/HU
Martín	[HU-01][RF-01] Solicitud pública de inscripción	HU-01 / RF-01

Responsable	Issue	RF/HU
Martín	[RF-04] Aprobar y habilitar alumno	RF-04
Claudio	[HU-04][RF-05] Registrar profesor	HU-04 / RF-05
Claudio	[RF-15] Proteger rutas privadas por rol	RF-15
Compartido	[TEST] Smoke Sprint 1: alumno + profesor + auth	N/A

SPRINT 2

Responsable	Issue	RF/HU
Martín	[RF-08/RF-09] Deportes y grupos deportivos	RF-08 / RF-09
Martín	[HU-02][RF-10] Inscripción a deporte	HU-02 / RF-10
Claudio	[HU-05][RF-11] Inscripción a transporte	HU-05 / RF-11
Compartido	[TEST] Smoke Sprint 2: deporte + transporte	N/A

SPRINT 3

Responsable	Issue	RF/HU
Martín	[RF-14] Asociación padre-hijo	RF-14
Martín	[RF-18] Materias mínimas del curso	RF-18
Martín	[HU-03][RF-20] Inscripción al cursado	HU-03 / RF-20
Claudio	[RF-21] Profesor-materia-curso mínimo	RF-21
Claudio	[HU-06][RF-22] Reportes institucionales	HU-06 / RF-22
Compartido	[TEST] Smoke Sprint 3: cursado + reportes	N/A

CIERRE

Responsable	Issue	RF/HU
Martín	[CLEANUP] Eliminar mocks/localStorage de sus HU	HU-01 / HU-02 / HU-03
Claudio	[SECURITY] Revisar Auth, RLS y permisos	RF-13 / RF-15
Claudio	[CLEANUP] Eliminar fallbacks demo compartidos	N/A
Compartido	[REGRESSION] Probar las 6 HU priorizadas	HU-01 a HU-06

Responsable	Issue	RF/HU
Compartido	[RELEASE] Deploy Vercel + Supabase + demo final	N/A

11. Requerimientos restantes - Backlog posterior

Estos RF tienen dueño definido, pero no deben desplazar las seis HU prioritarias antes de la entrega web salvo que una dependencia técnica los haga necesarios.

Responsable	Backlog
Martín	RF-02 Consulta alumno; RF-03 Modificación alumno; completar RF-16/17/19 con gestión; RF-23 Chatbot deportivo.
Claudio	RF-06 Consulta profesor; RF-07 Modificación profesor; RF-12 Comedor; completar RF-13/15 y RF-21 según alcance final.

12. Reglas para no pisarse - humanos y Codex

- Un Issue = un responsable = una rama.
- Nadie hace commits directos a main ni a la rama de integración.
- Martín no modifica features de Auth/Profesores/Transporte/Reportes sin handoff. Claudio no modifica Alumno/Deportes/Cursado sin handoff.
- App.tsx, package.json, src/shared/**, contratos comunes y config Supabase son zona compartida: PR pequeño y revisión del otro.
- Codex debe recibir el Issue completo y trabajar solo dentro de ese alcance; no refactorizar áreas no relacionadas.
- Si falta una dependencia: primero contrato + MOCK/SEED documentado. Esperar a que el otro termine código no es una estrategia de trabajo.
- Una migración ya mergeada no se edita; se crea una migración nueva.
- No subir .env, service_role keys ni contraseñas reales.

Prompt mínimo para Codex

“Respetá AGENTS.md y el Issue actual. No modifiques archivos fuera del alcance. Si necesitás tocar una zona compartida, cambiar un contrato o crear un mock, explicá primero el motivo y dejalo documentado en el PR.”

13. Definition of Done simplificada

- Criterios de aceptación cumplidos.
- Código mergeado mediante PR revisado por el compañero.
- Persistencia real en Supabase para la HU de entrega.
- RLS/constraints/reglas críticas aplicadas cuando corresponde.
- Pruebas mínimas ejecutadas.
- No quedan mocks productivos ocultos.
- Issue y GitHub Project actualizados.

14. Calendario operativo hasta fin de septiembre

Ventana	Objetivo
26-30 agosto	Fundación técnica + contratos + actualización del Plan de Trabajo.
31 ago - 6 sep	Sprint 1 revisado: HU-01 + RF-04 / HU-04 + Auth base.
7-13 sep	Sprint 2 revisado: HU-02 deportes / HU-05 transporte.
14-20 sep	Sprint 3 revisado: HU-03 cursado / HU-06 reportes.
21-26 sep	Hardening: RLS, E2E, responsive, eliminar mocks/fallbacks.
27-30 sep	Code freeze, deploy, evidencia y demo.

Actualización documental necesaria

El Plan de Trabajo actual debe modificarse para que el orden de Sprints coincida con este nuevo orden operativo. No conviene mantener dos cronologías distintas. La app móvil continúa fuera del desarrollo activo hasta octubre y consumirá el mismo Supabase.

15. Fuentes y trazabilidad

- TP-Metodologia-2.pdf: alcance general, módulos, reportes y reglas de negocio.
- Plan_de_Trabajo_con_Piramide.docx: RF-01 a RF-23, HU-01 a HU-06, criterios de aceptación, tecnologías y arquitectura.
- Plan operativo anterior: reglas de ramas, ownership, mocks, Supabase y Codex que se simplifican y reordenan en esta versión.
- Decisión del equipo del 26/08/2026: actualizar el orden de Sprints para respetar dependencias lógicas y reducir mocks.

Regla final

Primero datos base, después inscripciones, al final reportes. Cada RF tiene un dueño. Las dependencias se resuelven con contratos y, solo cuando sea necesario, con MOCK/SEED documentado. Esa es la guía principal para GitHub Projects, Git y Codex.